

TP4

Identification des objets recyclables

Présentation :

Dans le cadre de la protection de l'environnement et pour des fins d'amélioration de la qualité de vie des citoyens de la ville de **Bizerte**, des jeunes diplômés de la spécialité « **Data Science** » mettent en place une startup de valorisation de déchets. Une tâche primordiale de ce projet promoteur, consiste à robotiser le processus. Ceci fait appel aux compétences de la vision intelligente notamment l'identification des objets recyclable. C'est dans ce cadre qu'entre le présent TP.

- Remarque : l'objectif de ce TP est de pratiquer les notions de base de réseau convolutif profond (CNN). Des solutions plus sophistiquées sont envisageable.
- Nous utilisons certains bibliothèques Python pour réaliser le travail demandé
 - o Module de traitement d'image « Skimage » : <https://scikit-image.org/>
 - o Module de gestion des figures « Matplotlib » : <https://matplotlib.org/>

Objectif :

- Préparer et explorer la base des images
- Augmenter la taille des bases
- Structurer un projet « machine learning »
- Concevoir un réseau de neurones convolutif profond
- Implémentation via le Framework « Keras »
- Tester les performances du réseau en fonction des hyper-paramètres
- Choix des hyper-paramètres optimaux

Travail demandé

Le projet envisagé fait incorporer plusieurs composantes et compétences notamment la robotique, l'intelligence artificielle, la vision intelligente, ...l'équipe « R+ » formée par Rania Swissi et ses collègues a été chargée de mettre en place une technique de vision intelligente afin de classifier les objets recyclables (papier, bouteille en plastique, canette en aluminium) qui seront par la suite trier systématiquement. La solution adoptée par « R+ » consiste à développer un réseau de neurones convolutif profond (CNN).



Bouteille en plastique



Canette en aluminium



Papier

Partie 1 : Préparation des données

- 1) « R+ » a été chargée de préparer une base de données afin d'en servir pour entrainer et tester les modèle CNN. La base collectée est formée par des images relatives aux trois classe (1) papier (2) bouteille en plastique (3) canette en aluminium. On vous donne la base « waste_classification » sous format zippé.
 - a. Explorer la base (structure de dossier, type d'images, taille d'image en bits)
 - b. La structure de la base fournie par « R+ » n'est pas adéquate au projet de classification. Dites pour quoi ?
 - c. Proposer une structure de projet qui permet présenter le projet d'une façon claire.

- 2) Ecrire un code Python qui permet de respecter la structure proposée et de créer les répertoires nécessaires.

Indication : utiliser les méthodes du module `os` : `getcwd()`, `join.path()`, `makedirs()`

- 3) Après avoir créé les répertoires selon l'architecture proposée, on procède à la sélection des images de la base fournie par « R+ » et le copiage vers les dossiers créés précédemment.
 - a. Explorer et visualiser les images du dossier (Waste_classification/DATASET/TRAIN/R)
 - b. On s'intéresse uniquement aux images dont leurs noms contiennent le terme « **plastic** ». On cherche à copier ces images vers la nouvelle structure du projet c-à-d vers le dossier (base/app/plastique). Pour le faire, on vous donne ce bout de programme. Tester le et vérifier son fonctionnement et résultat.

```
import shutil
scr = os.path.join(swd, 'Waste_classification/DATASET/TRAIN/R')
dest = path = os.path.join(swd, 'base/app/plastique');
os.chdir('./Waste_classification/DATASET/TRAIN/R')
R_im = os.listdir()
shutil.copy(scr+'plasticbottle (48).jpg', dest+'plasticbottle (48).jpg')
```

- c. En s'inspirant du code précédent, écrire un code qui permet de copier toutes les images dont leurs noms contiennent le terme « plastic » vers le dossier (base/app/plastique)
 - d. Faire de même pour le reste de la base (Waste_classification) afin de séparer les objets à classier en trois groupe (1) papier (2) bouteille en plastique (3) canette en aluminium et respectant l'architecture du projet proposée.
- 4) De la même façon préparer la base de validation
- 5) Mise à l'échelle des images

- a. Visualiser quelques images (3 ou plus) de la base (base/app/canette) et afficher leurs tailles.

Indication : utiliser la bibliothèque « `skimage` » et importer les le module « `io` ». Utiliser notamment les fonction `imread()` et `imshow()`

- b. Durant la phase d'apprentissage, nous aurons besoin de mettre à l'échelle les images d'apprentissage pour avoir la même taille.
 - i. A l'aide du la fonction « `resize()` » de « `skimage.transform` » mettre à l'échelle une image pour avoir une taille (64, 64, 3)
 - ii. Afficher sur la même figure l'image avant et après transformation. Interpréter
 - c. Nous désirons mettre à l'échelle toutes les images du répertoire (base/app/canette). Pour cela, copier une image de type « canette » sur le répertoire principal afin de faire vos tests préliminaires. Mettre à l'échelle l'image (64, 64, 3) puis écraser l'ancienne image par

la nouvelle avec la taille désirée. Il faut enregistrer l'image tels que ses pixels sont des entiers sur 8 bits (unit8) à valeurs entre [0, 255]

Indication :

```
img_resized = resize(img, (Nx, Ny, Nz))
img_resized = img_resized * 255 # conversion --> [0, 255]
img_resized = img_resized.astype(np.uint8)
imsave(im, img_resized) # ecrase l'ancienne image par la nouvel resized
```

- d. Développer un code pour mettre à l'échelle toutes les images des deux bases
« apprentissage » et « test »

Partie 2 : Conception du réseau CNN

- 1) « R+ » pense utiliser le Framework « Keras » pour implémenter une première solution au problème de classification. Keras dispose de 3 méthodes pour entrainer un réseau profond :

```
.fit
.fit_generator
.train_on_batch
```

R+ opte pour la première méthode qui télécharge toute la base dans la RAM du processeur et qui nécessite une structure de données spécifique. Dans le cas du présent projet, « R+ » doit mettre les données sous forme d'un tableau « numpy » $X_{train} : (Nx, dim1, dim2, dim3)$ où Nx désigne le nombre des images de la base. $dim1$, $dim2$ et $dim3$ désignent les dimension des images. Les labels sont sous forme d'un tableau numpy 1D.

- a. Déplacer 04 images de la base d'apprentissage dans le répertoire principal. Lire les images et mettre les images sous forme d'un tableau numpy (tenseur 4D) c.-à-d. (4, 64, 64, 3) .

Indication : *from skimage.io import imread*
 $img1 = np.expand_dims(img1, axis=0)$
 $images = np.concatenate((img1, img2), axis=0)$

- b. Écrire un code qui permet de mettre en forme les données d'apprentissage et de validation selon la structure requise.

Indication : *utiliser la fonction ci-dessous*

```
# ***** fonctions utiles *****
def mise_forme(path_base):
    os.chdir(path_base)
    images_list = os.listdir()
    numb_images = len(images_list)
    images = np.empty((numb_images, 64, 64, 3), dtype=np.uint8)
    k = 0
    for im in images_list:
        img = imread(im)
        img = np.expand_dims(img, axis=0)
        images[k, :, :, :] = img
        k += 1
    return(images)
# *****
```

- c. Créer pour chaque classe d'image l'étiquette de la classe.
Exemple : `y_apprentissage_canette` est un tableau numpy de dim $(N_c, 1)$. Avec N_c est le nombre d'image de type canette.
- d. Arranger toutes les images d'apprentissage sous forme d'un tenseur 4D et les étiquettes relatives sous forme d'un tenseur 1D.
- e. Enregistrer dans le répertoire principal les deux tenseurs sous forme de deux fichiers de format « .npy »

Indication : `np.save('X_app.npy', X_app)`

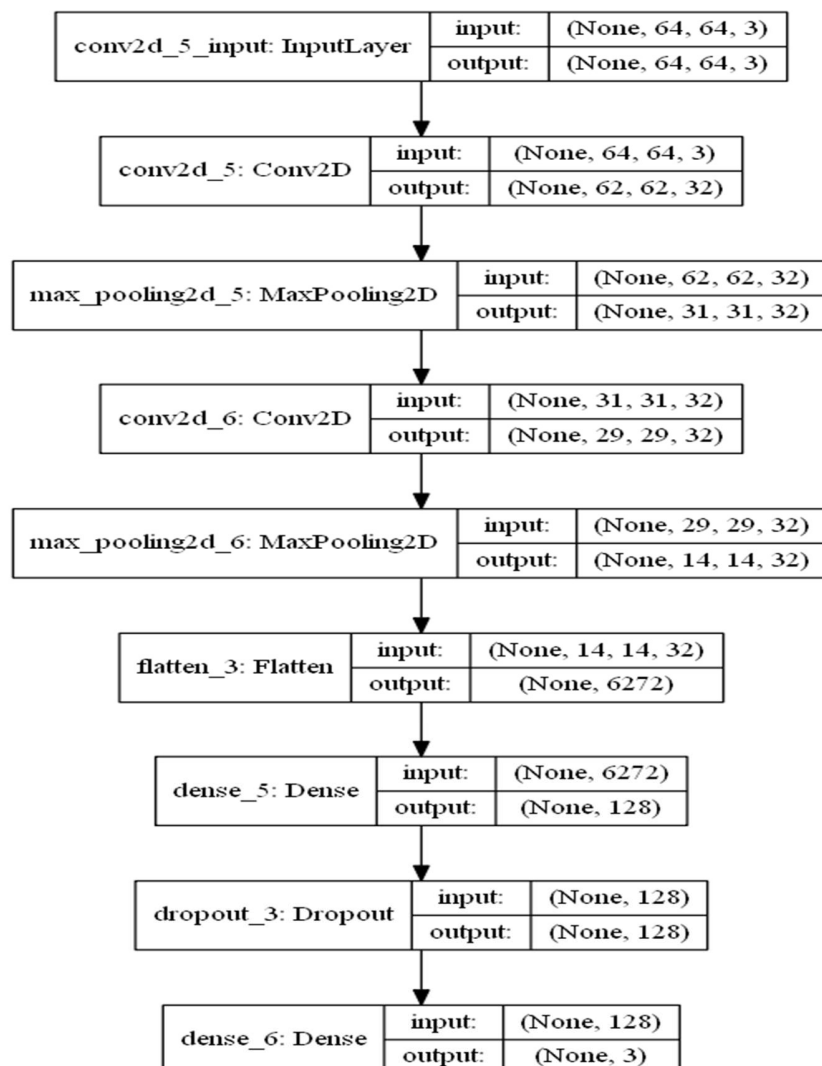
- f. Faire de même pour la base de validation.

2) A ce stade-là, les données sont bien mises en forme. « R+ » commence alors à implémenter en propre la solution proposée basée sur les réseaux convolutifs profond.

- a. Télécharger les données

Indication : `X_app = np.load('X_app.npy')`

- b. Faites de sorte que les données seront mises à l'échelle telles que les valeurs des pixels seront dans $[0, 1]$.
- c. Concevoir un réseau CNN dont sa topologie est représentée ci-dessous. Afficher le résumé du modèle et sa topologie.



- d. Quel est le rôle de la couche « dropout »
- e. On s'intéresse à la première couche de convolution

conv2d_5: Conv2D	input:	(None, 64, 64, 3)
	output:	(None, 62, 62, 32)

- i. On remarque que la sortie de la couche conv2D est de taille (62, 62, 32) alors que l'entrée est une image de taille (64, 64, 32). Expliquer cette variation de taille.
 - ii. A votre avis quel est la taille des filtres de convolution
 - iii. Quel est le nombre de ces filtres
- f. On s'intéresse à la couche de « pooling »

max_pooling2d_5: MaxPooling2D	input:	(None, 62, 62, 32)
	output:	(None, 31, 31, 32)

- i. Expliquer les sorties et les entrées de ladite couche
 - ii. Quelles est la taille du filtre de pooling
 - iii. A quoi sert le pooling
 - g. Quelle est la fonction d'activation de la dernière couche. Justifier votre réponse.
 - h. Déterminer le nombre total des paramètres à entraîner.
 - i. Faire un schéma à main levée du réseau CNN
- 3) Compiler le modèle tel que :
- loss = crosse entropy categoriel
 - optimizer = l'algorithme Adadelta
 - metrics = accuracy
- 4) Apprentissage
- a. Entraîner le modèle pour un nombre d'époch = 10 et un mini batch = 64.
 - b. Sur la même figure, tracer l'évolution de la fonction perte et l'accuracy en fonction des epochs pour les deux bases : apprentissage et validation.
Indication : utiliser l'attribut « history » du modèle entraîné
 - c. Augmenter le nombre d'épochs puis re-simuler pour avoir des résultats plus significatifs.
- 5) Validation
- Evaluer le modèle entraîné sur la base de validation et déterminer le score obtenu (accuracy)
- Indication : score = model.evaluate(valX, valY, verbose=1)*
- 6) Test
- a. Charger la base de test par quelques images
 - b. Tester le modèle sur la base de test
 - c. Interpréter
- 7) Prédiction
- a. Préparer une image selon la structure du projet
 - b. Prédire sa classe
Indication : predict_classes = model.predict_classes(valX)